# Efficient Token Compression for the Understanding and Generation Unified MLLMs

Junyan Lin[1] Jinming Liu[1] Shengyang Zhao[1] Xin Jin[1,2,†]
[1]Eastern Institute of Technology, Ningbo, China
[2]Zhongguancun Academy, Beijing, China
linjyan00@gmail.com, jinxin@eitech.edu.cn

***Abstract*—Recent advances in multimodal large language models (MLLMs) leverage a single tokenizer for both visual understanding and generation, unifying perception and synthesis within one architecture. However, to effectively perform both generation and comprehension tasks, existing methods typically rely on a large number of tokens for reasoning, which significantly hinders practical applications. Thus, a general token compression strategy is urgently required to improve MLLMs' efficiency. In this work, we systematically examine how token compression affects model performance. We classify the mainstream tokenizers into three categories according to their different focuses: Semantic-biased, Detail-biased, and Balanced. Then, we apply three potential ways for compression at the beginning, intermediate, and final stages. Through extensive experiments, we get some interesting findings: *Detail-biased tokenizers are robust to compression at the input stage but highly sensitive on the final stage, while Balanced tokenizers inherit semantic robustness that mitigates compression degradation for the final representation.* These insights inspire us to design a dual-branched Balanced tokenizer, whereas we conduct compression for the Detail-biased branch at the beginning as well as for the Semantic-biased branch at the ending, achieving a win-win effect.**

## I. Introduction

Multimodal large language models (MLLMs) [1]–[5] are increasingly converging toward a unified paradigm for visual understanding and generation, which is widely regarded as a promising pathway to Artificial General Intelligence (AGI). This unified approach enables models to seamlessly integrate perception and generation within a shared cognitive framework. Earlier methods separated these abilities into two specialized tokenizers, typically a CLIP-style encoder [6], [7] for understanding and a VAE/diffusion encoder for generation [8]–[10], resulting in considerable complexity, duplicated parameters, and heavy computational overhead. In contrast, recent work [4], [5], [11] feeds one sequence of visual tokens into the LLM, thus reducing the memory cost.

Although different unified tokenizers share this high-level goal, they differ in the type and granularity of information each token carries. We group existing designs into three archetypes. A *Semantic-biased* tokenizer such as BLIP-3/3o [12], uses a CLIP-ViT encoder [6] that compresses an image into a handful of visual patch tokens. These tokens align well with text but inevitably discard fine texture. A *Detail-biased* tokenizer, such as the VAE encoder [13] in Emu-3 [4], produces hundreds of latent tokens that preserve local appearance while exhibiting weaker textual alignment. Sitting between the two, a *Balanced* tokenizer, for instance Show-o2 [5], delivers text-aligned semantic tokens alongside appearance-rich detail tokens, seeking a balanced trade-off between understanding accuracy and generative fidelity.

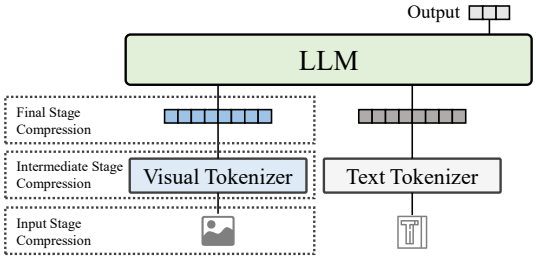


Fig. 1. Compression Stages in MLLMs. We restrict our focus to compression strategies outside the LLM, acting only on the output of the visual tokenizer or its preceding stages.

Regardless of the tokenizer type, the length of the visual token sequence governs both GPU memory footprint and inference latency, as self-attention scales quadratically with token length. To handle these problems, recent studies have begun to explore visual token compression (VTC), which aims to shorten the visual token sequence. As shown in Fig. 1, VTC can be inserted at three distinct stages in the visual pipeline: Beginning Stage, where raw visual inputs are downsampled before tokenization; Intermediate Stage, which compresses token representations within the visual tokenizer; and Final Stage, where the output token sequence of the tokenizer is compressed before being fed into the LLM.

Almost all prior studies target Semantic-biased tokenizers, leaving the behavior of Detail-biased and Balanced tokenizers largely unexplored. In this paper, we address this gap through the first systematic comparison of VTC strategies across all three tokenizer families in a training-free situation. We benchmark representative insertion stages, quantify their impact on understanding-centric benchmarks, and reveal that Detail-biased tokenizers handle input stage compression well but are vulnerable to compression within or after the tokenizer. In contrast, Balanced tokenizers struggle with early compression

979-8-3315-7769-8/26/$31.00 ©2026 IEEE

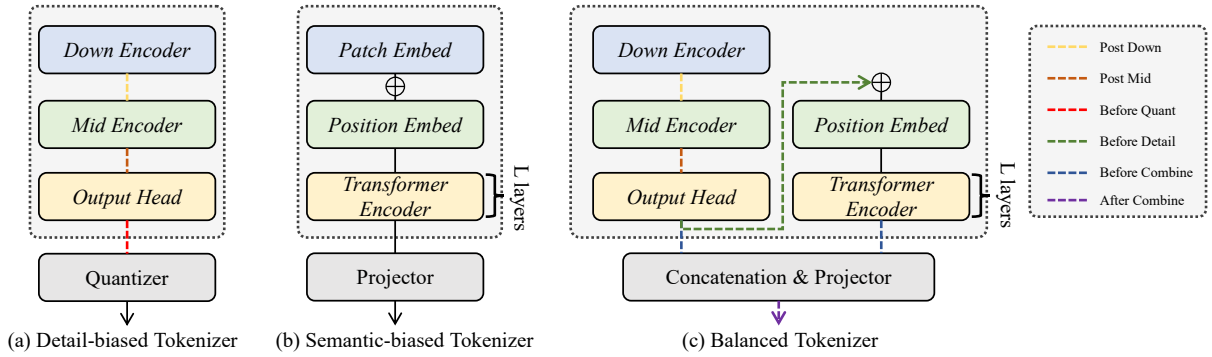


Fig. 2. Comparison of Visual Tokenizer Structures. (a) The Detail-biased tokenizer preserves fine-grained visual features via a multi-stage encoder followed by quantization. (b) The Semantic-biased tokenizer extracts high-level semantics using a transformer-based encoder with patch and position embeddings. (c) The Balanced tokenizer integrates both detail and semantic branches, combining outputs through a unified projection for comprehensive representation.

due to positional encoding sensitivity but remain more robust at the final stage. These findings suggest a potential direction: compress the detail path early and the semantic path later to balance efficiency and performance.

.

## II. Visual Token Compression in MLLMs

The basic goal of VTC is to shorten the visual token sequence while minimizing damage to downstream performance. Current methods can be broadly divided into two main approaches, Selection and Merging. *Selection* approaches rank tokens, often by attention saliency as in FastV [14] or by learnable dropout probabilities as in PDrop [15], and then prune the least informative visual tokens. *Merging* approaches cluster similar or neighboring tokens and replace each cluster with an aggregate representation, as implemented by ToMe [16], DeCo [17], and C-Abstractor [18]. While selection approaches are efficient and easy to implement, they may eliminate complementary visual cues distributed across low-saliency tokens and incompatible with attention implementations such as FlashAttention. Merging approaches preserve broader contextual coverage, but can dilute fine-grained discriminative features needed for accurate understanding. Although these methods have shown promising results, they are predominantly designed and evaluated on Semantic-biased tokenizers or performed compression *inside* the LLM itself, making it hard to untangle tokenizer-specific effects. We therefore restrict our study to *pre-LLM, training-free* compression, allowing a clean comparison across different tokenizers.

## III. Unified Tokenizer Framework

In this section, we first introduce three representative unified tokenizers, namely the Detail-biased tokenizer, the Semantic-biased tokenizer, and the Balanced tokenizer. We provide an in-depth discussion of their architectural designs, focusing on how each type captures visual information at varying granularity levels. Understanding these structural differences is essential for exploring their distinct behaviors under token compression strategies, which we will discuss extensively in the subsequent sections.

### A. Structure of Different Tokenizers

*a) Detail-biased Tokenizer:* focuses on preserving fine-grained local visual information. A typical implementation is based on image generative models such as VQ-GAN [19]. As illustrated in Fig. 2(a), the structure generally includes three encoder stages: the Down Encoder performs initial downsampling and low-level feature extraction; the Mid Encoder further processes intermediate representations; and the Output Head generates rich visual features. These features are then quantized via a codebook-based Quantizer into discrete visual tokens. This quantization process enables the transformation of continuous visual representations into discrete symbolic tokens, analogous to words in language models.

*b) Semantic-biased Tokenizer:* focuses on abstracting high-level semantic concepts. As shown in Fig. 2(b), the structure typically consists of four components: the input image $I \in \mathbb{R}^{H\times W\times 3}$ is first processed by a Patch Embedding module, followed by the addition of Position Embedding to encode spatial information. The resulting tokens are passed through a Transformer Encoder [20] with $L$ layers for semantic modeling. Finally, the output is projected into a shared semantic space via a Projector to support multimodal alignment with large language model.

*c) Balanced tokenizer:* By integrating Detail-biased and Semantic-biased tokenizers, Balanced tokenizers can yield more comprehensive representations that capture both low-level visual features and semantic information. Although there is no fixed paradigm yet, one representative example is Show-o2 [5]. As shown in Fig. 2(c), the input image is first processed by a Detail-biased branch (including Down Encoder, Mid Encoder, and Output Head) to extract fine-grained features. These are then passed into a Semantic-biased branch (including Position Embedding and Transformer Encoder) to further extract semantic information. The outputs of both branches are concatenated and fed into a projector to obtain unified representations.

TABLE I

METRICS UNDER INPUT COMPRESSION (1× VS 2×) ACROSS BENCHMARKS. RED ARROWS INDICATE PERFORMANCE DROPS RELATIVE TO THE 1× SETTING, AND TEAL ARROWS INDICATE GAINS.

| Model | AI2D | ChartQA | DocVQA | InfoVQA | MMBench-EN | OK-VQA | TextVQA | VizWiz | VQAv2 |
|---|---|---|---|---|---|---|---|---|---|
| Emu-3 (1×) | 67.80 | 58.20 | 73.23 | 37.36 | 76.52 | 37.20 | 68.24 | 23.34 | 66.54 |
| Emu-3 (2×) | 63.20↓7% | 23.00↓60% | 45.06↓38% | 26.00↓30% | 71.21↓7% | 32.08↓14% | 48.36↓29% | 24.02↑3% | 59.30↓11% |
| Show-o2 (1×) | 73.20 | 50.00 | 64.64 | 43.48 | 84.85 | 58.00 | 67.34 | 52.20 | 75.36 |
| Show-o2 (2×) | 62.00↓15% | 15.20↓70% | 9.63↓85% | 20.45↓53% | 58.33↓31% | 18.80↓68% | 13.88↓79% | 39.34↓25% | 44.48↓41% |

TABLE II

METRICS ACROSS INTERMEDIATE STAGE COMPRESSION (**2× MEAN POOLING**). PD DENOTES POST DOWN, AND PM DENOTES POST MID.

| Model Variant | AI2D | ChartQA | DocVQA | InfoVQA | MMBench-EN | OK-VQA | TextVQA | VizWiz | VQAv2 |
|---|---|---|---|---|---|---|---|---|---|
| Emu-3 | 67.80 | 58.20 | 73.23 | 37.36 | 76.52 | 37.20 | 68.24 | 23.34 | 66.54 |
| Emu-3 (PD) | 59.80↓12% | 15.00↓74% | 8.75↓88% | 19.58↓48% | 64.39↓16% | 26.72↓28% | 19.74↓71% | 20.36↓13% | 50.64↓24% |
| Emu-3 (PM) | 58.40↓14% | 12.40↓79% | 6.87↓91% | 18.12↓52% | 53.03↓31% | 19.56↓47% | 13.28↓81% | 16.36↓30% | 41.30↓38% |
| Show-o2 | 73.20 | 50.00 | 64.64 | 43.48 | 84.85 | 58.00 | 67.34 | 52.20 | 75.36 |
| Show-o2 (PD) | 60.00↓18% | 14.80↓70% | 8.82↓86% | 19.61↓55% | 52.27↓39% | 14.60↓75% | 8.80↓87% | 38.42↓27% | 41.08↓45% |
| Show-o2 (PM) | 60.00↓18% | 14.80↓70% | 8.66↓87% | 20.46↓53% | 51.52↓39% | 14.08↓76% | 8.78↓87% | 38.28↓27% | 40.88↓46% |

TABLE III

METRICS ACROSS FINAL STAGE COMPRESSION STRATEGIES (**2× MEAN POOLING**). BQ DENOTES BEFORE QUANT, AD DENOTES AFTER DETAIL, BC DENOTES BEFORE COMBINE, AND AC DENOTES AFTER COMBINE THE BRANCHES.

| Model Variant | AI2D | ChartQA | DocVQA | InfoVQA | MMBench-EN | OK-VQA | TextVQA | VizWiz | VQAv2 |
|---|---|---|---|---|---|---|---|---|---|
| Emu-3 | 67.80 | 58.20 | 73.23 | 37.36 | 76.52 | 37.20 | 68.24 | 23.34 | 66.54 |
| Emu-3 (BQ) | 56.80↓16% | 12.60↓78% | 7.32↓90% | 16.31↓56% | 51.52↓33% | 18.56↓50% | 12.10↓82% | 14.24↓39% | 40.60↓39% |
| Show-o2 | 73.20 | 50.00 | 64.64 | 43.48 | 84.85 | 58.00 | 67.34 | 52.20 | 75.36 |
| Show-o2 (AD) | 61.40↓16% | 14.20↓72% | 8.11↓88% | 19.28↓56% | 50.76↓40% | 15.48↓73% | 7.92↓88% | 38.22↓27% | 40.08↓47% |
| Show-o2 (BC) | 64.20↓12% | 20.20↓60% | 24.26↓62% | 26.35↓39% | 78.03↓8% | 30.28↓48% | 38.44↓43% | 44.38↓15% | 58.74↓22% |
| Show-o2 (AC) | 61.80↓16% | 19.20↓62% | 20.33↓69% | 24.11↓45% | 71.97↓15% | 27.60↓52% | 32.34↓52% | 42.10↓19% | 54.84↓27% |

## IV. EXPERIMENTS

### A. Experimental Setup

To systematically analyze VTC across different tokenizer families, we design a controlled evaluation protocol. This setup ensures fair comparison while avoiding the cost of retraining Detail-biased tokenizers, which typically require massive data. We adopt 2× mean pooling as the default compression operator throughout our experiments.

*a) Tokenizer Scope:* We focus our experiments on Detail-biased and Balanced tokenizers. **Semantic-biased tokenizers are not included**, as prior works [17], [30], [31] have already extensively explored their compression behavior. By contrast, the compression characteristics of Detail-biased and Balanced tokenizers remain largely underexplored. **For evaluating Detail-biased and Balanced tokenizers, we adopt Emu-3 and Show-o2, respectively.** [1]

[1]Official implementations: https://github.com/baaivision/Emu3 and https://github.com/showlab/Show-o

### B. Compression Stages

Based on where the compression is applied, we categorize visual token compression (VTC) into three stages: Input, Intermediate, and Final. This structured division enables a systematic investigation of how different compression strategies affect tokenizer performance in multimodal tasks.

*a) Input Stage Compression:* refers to reducing the spatial resolution of the input image before it is fed into the tokenizer. For CNN-based Detail-biased tokenizers, compression can be implemented directly by resizing the input image (e.g., downscaling from $H \times W$ to $(H/2) \times (W/2)$). For transformer-based Semantic-biased tokenizers, positional embeddings must be interpolated to adapt to the new resolution and maintain positional consistency in the token sequence.

*b) Intermediate Stage Compression:* targets intermediate stages within the tokenizer. For Detail-biased tokenizers (as illustrated in Fig. 2(a)), compression can be introduced after the Down Encoder (`Post Down`) or Mid Encoder (`Post Mid`) by inserting compressing modules, thereby reducing subsequent computational requirements. For transformer-based

tokenizers, compression modules can be applied across transformer layers to progressively reduce token count. Specifically, for Detail-biased and Balanced tokenizers in our experiments, we compress features at two strategic positions: `Post Down` and `Post Mid`.

*c) Final Stage Compression:* is applied to the tokenizer's output to reduce token count before passing them to the large language model. For Detail-biased tokenizers producing discrete visual tokens, compression can occur prior to quantization (`Before Quant`). For Semantic-biased and Balanced tokenizers outputting continuous sequences, operations like mean pooling can aggregate sequences, preserving semantic integrity while reducing token count. In our evaluations, we examine three specific variants for Balanced tokenizers: (1) compressing only Detail-biased tokens before entering the Semantic-biased branch (`After Detail`), (2) independently compressing outputs from both branches prior to their combination (`Before Combine`), and (3) jointly compressing the combined outputs of both branches (`After Combine`).

### C. Benchmarks

We evaluate our models on ten widely-used vision–language benchmarks: AI2D [21], ChartQA [22], DocVQA [23], InfoVQA [24], MMBench-EN [25], OK-VQA [26], TextVQA [27], VizWiz-VQA [28], and VQAv2 [29]. Each split contains 500 examples pre-selected by the lmms-lab/LMMs-Eval-Lite suite, and we use them as provided without additional filtering.

### D. Results & Analysis

*a) Input Stage Compression Performs Well for Detail-biased Tokenizers:* **Detail-biased model (Emu-3):** As shown in Table I, for Emu-3, reducing the input resolution by 2× results in only mild accuracy degradation in most benchmarks. In some benchmarks such as VizWiz, performance even slightly improves (23.34% to 24.02%, ↑3%). This robustness is attributed to Emu-3's CNN-based tokenizer, which preserves spatial inductive bias and generalizes well to low-resolution inputs. **Balanced model (Show-o2):** Show-o2 incorporates a transformer-based semantic branch, which is sensitive to resolution changes. With 2× input compression, Show-o2 experiences catastrophic performance drops. This severe degradation stems from the model's fixed position embeddings, which are not resolution-invariant, and the lack of re-training under the new input scale. These effects are especially pronounced in text-centric and document-heavy tasks.

*b) Compression Inside the Tokenizer Is Not Viable:* As shown in Table II, Despite the relative robustness of Balanced models, Intermediate stage compression introduces substantial performance degradation, with accuracy drops exceeding 70% on several benchmarks. This indicates that Intermediate stage compression is fundamentally flawed and should not be considered a viable option for visual token compression.

*c) Balanced Tokenizers Inherit Semantic Robustness in Final Stage Compression :* Table III reveals that compressing in Final stage yields divergent effects on Emu-3 and Show-o2.

**Detail-biased model (Emu-3):** The performance collapses across all benchmarks under Final stage compression. As shown by the `Before Quant` configuration, accuracy drops exceed 80% on tasks like DocVQA and TextVQA. This sharp degradation highlights the vulnerability of Detail-biased representations: once the visual detail-rich tokens are extracted, compressing them at the end-stage removes critical fine-grained information with no opportunity for later recovery. The absence of semantic abstraction layers leaves the model overly dependent on uncompressed low-level features, making final-stage compression particularly destructive.

**Balanced model (Show-o2):** Show-o2 demonstrates significantly more stable behavior. Both `Before Combine` and `After Combine` configurations show moderate degradation. However, when compression is applied immediately after the detail branch (`After Detail`), Show-o2 exhibits severe degradation, mirroring the failure patterns seen in Emu-3. This underscores that the semantic branch is essential in preserving robustness: multiple works [17], [30], [31] have shown that semantic embeddings are resilient to Final stage compression. When the final representation involves a fusion of both detail and semantic features, compression allows the semantic branch to compensate for the loss of detail granularity.

*d) Toward Compression-Aware Tokenizer Design: Leveraging Strengths at Both Ends:* Taken together, our findings highlight a critical insight: different tokenizer architectures exhibit complementary robustness at different compression stages. Specifically, Detail-biased tokenizers such as Emu-3 are robust to Input stage compression but highly vulnerable to post-tokenization compression, while Semantic-biased tokenizers are more tolerant to final Stage compression yet sensitive to early-stage resolution reduction.

This suggests a promising direction: future compression-aware designs should jointly leverage the Input stage resilience of Detail-biased tokenizers and the Final stage robustness of Semantic-biased representations. By combining the strengths of both components in a hybrid framework, compression can be enabled at both ends with minimal performance degradation. Such dual stage strategies may serve as an effective path towards efficient and scalable multimodal models.

## V. Conclusion

This paper presents a comprehensive study of visual token compression (VTC) in unified multimodal large language models, focusing on the underexplored Detail-biased and Balanced tokenizer architectures. Through a controlled, training-free experimental protocol, we evaluate the impact of compression at three Stages in the tokenization pipeline: Input, Intermediate, and Final stages. Our findings reveal striking differences across tokenizer types. Detail-biased tokenizers exhibit strong resilience to input compression due to their spatial inductive biases, but suffer severe performance drops when compressed at later stages. Balanced tokenizers, on the other hand, show greater tolerance to Final stage compression, thanks to their inclusion of Semantic-biased tokenizers that are inherently robust to token sparsity. These results highlight a

promising direction for future work lies in dual-stage designs that combine the Input stage robustness of Detail-biased components with the Final stage stability of Semantic-biased ones. Such approaches offer a path toward efficient and generalizable MLLMs under constrained computational budgets.

## ACKNOWLEDGMENTS

This work was supported by Grants of NSFC 62302246, ZJNSFC LQ23F010008, Ningbo 2023Z237 & 2024Z284 & 2024Z289 & 2023CX050011 & 2025Z038 & 2025Z059, and supported by High Performance Computing Center at Eastern Institute of Technology and Ningbo Institute of Digital Twin.

## REFERENCES

[1] H. Liu, C. Li, Q. Wu, et al. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36:34892–34916, 2023.

[2] S. Bai, K. Chen, X. Liu, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[3] H. Lu, W. Liu, B. Zhang, et al. Deepseek-vl: Towards real-world vision-language understanding. *arXiv preprint arXiv:2403.05525*, 2024.

[4] X. Wang, X. Zhang, Z. Luo, et al. Emu3: Next-token prediction is all you need. *arXiv preprint arXiv:2409.18869*, 2024.

[5] J. Xie, Z. Yang, and M. Z. Shou. Show-o2: Improved native unified multimodal models. *arXiv preprint arXiv:2506.15564*, 2025.

[6] A. Radford, J. W. Kim, C. Hallacy, et al. Learning transferable visual models from natural language supervision. *Proceedings of the International Conference on Machine Learning*, pages 8748–8763, 2021.

[7] X. Zhai, B. Mustafa, A. Kolesnikov, et al. Sigmoid loss for language image pre-training. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11975–11986, 2023.

[8] S. Wu, H. Fei, L. Qu, et al. Next-gpt: Any-to-any multimodal llm. *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[9] C. Deng, D. Zhu, K. Li, et al. Emerging properties in unified multimodal pretraining. *arXiv preprint arXiv:2505.14683*, 2025.

[10] X. Chen, Z. Wu, X. Liu, et al. Janus-pro: Unified multimodal understanding and generation with data and model scaling. *arXiv preprint arXiv:2501.17811*, 2025.

[11] Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.

[12] J. Chen, Z. Xu, X. Pan, et al. Blip3-o: A family of fully open unified multimodal models-architecture, training and dataset. *arXiv preprint arXiv:2505.09568*, 2025.

[13] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[14] L. Chen, H. Zhao, T. Liu, et al. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. *European Conference on Computer Vision*, pages 19–35, 2024.

[15] L. Xing, Q. Huang, X. Dong, et al. Pyramiddrop: Accelerating your large vision-language models via pyramid visual redundancy reduction. *arXiv preprint arXiv:2410.17247*, 2024.

[16] M. Kim, S. Gao, Y.-C. Hsu, et al. Token fusion: Bridging the gap between token pruning and token merging. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1383–1392, 2024.

[17] L. Yao, L. Li, S. Ren, et al. Deco: Decoupling token compression from semantic abstraction in multimodal large language models. *arXiv preprint arXiv:2405.20985*, 2024.

[18] J. Cha, W. Kang, J. Mun, et al. Honeybee: Locality-enhanced projector for multimodal llm. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13817–13827, 2024.

[19] P. Esser, R. Rombach, and B. Ommer. Taming transformers for high-resolution image synthesis. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12873–12883, 2021.

[20] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[21] T. Hiippala, M. Alikhani, J. Haverinen, et al. AI2D-RST: A multimodal corpus of 1000 primary school science diagrams. *Language Resources and Evaluation*, 55:661–688, 2021.

[22] A. Masry, D. X. Long, J. Q. Tan, et al. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. *arXiv preprint arXiv:2203.10244*, 2022.

[23] M. Mathew, D. Karatzas, and C. V. Jawahar. Docvqa: A dataset for vqa on document images. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 2200–2209, 2021.

[24] M. Mathew, V. Bagal, R. Tito, et al. Infographicvqa. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1697–1706, 2022.

[25] Y. Liu, H. Duan, Y. Zhang, et al. Mmbench: Is your multi-modal model an all-around player? *European Conference on Computer Vision*, pages 216–233, 2024.

[26] K. Marino, M. Rastegari, A. Farhadi, et al. Ok-vqa: A visual question answering benchmark requiring external knowledge. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3195–3204, 2019.

[27] A. Singh, V. Natarajan, M. Shah, et al. Towards vqa models that can read. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8317–8326, 2019.

[28] D. Gurari, Q. Li, A. J. Stangl, et al. Vizwiz grand challenge: Answering visual questions from blind people. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3608–3617, 2018.

[29] Y. Goyal, T. Khot, D. Summers-Stay, et al. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6904–6913, 2017.

[30] Y. Shang, M. Cai, B. Xu, et al. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024.

[31] S. R. Alvar, G. Singh, M. Akbari, et al. Divprune: Diversity-based visual token pruning for large multimodal models. *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 9392–9401, 2025.